MyCodeSchool

Code review — python-initiere

Runde de review pe proiectul tău, David, cea mai nouă sus.

Runda 2 — 2026-11-03 — sha 3d0360e

Stare: fișierele se compilează (`python -m py_compile cifre.py main.py` trece) — prima rundă în care nu te oprește o eroare de sintaxă. **4 din 5 🔴 ale rundei 1 sunt închise**: indentarea, `nr++` , `//` , numele greșit la apel. A rămas deschis cazul `n = 0` .

Verificat prin rulare: fiecare funcție pe valorile din fișă, plus stilul A comparat cu stilul B pe 5000 de numere aleatoare până la 1 000 000 000 (`suma_cifrelor` , `numar_cifre` , `palindrom` , `cifre_pare`) — nicio diferență în afară de cea din B2.

Terminat: problemele 1, 2, 3, 5, 6, 7 + problema 4 pe jumătate (doar stilul B). **Rămâne:** 4 stil A, 8 (fracția ireductibilă), 9 (număr perfect), 10 (cifra de control), 11 (cifrele impare dublate).

🟢 **Ce ai făcut bine, și nu e puțin:** `cifre_pare` și `sterge_cifra` sunt corecte din prima, inclusiv partea grea — cifrele ies în ordinea din numărul original (`nr = uc * p + nr` cu `p` înmulțit cu 10 **doar** când păstrezi o cifră) — și inclusiv cazul „nu rămâne nicio cifră” → `-1` , pe care l-ai prins singur, cu `p == 1` . Asta e exact gândirea pe care la matrici o scriai abia după ce ți-o arătam.

🔴 Critice

B1 🔴 `test_sterge_cifra()` crapă după ce afișează rezultatul corect — `cifre.py:155–163`

CUM E ACUM: apelez `test_sterge_cifra()` , tastez `1213415` și `1` . Afișează `2345` și `2345` — corect —, apoi: `UnboundLocalError: cannot access local variable 'ogl' where it is not associated with a value` (`cifre.py:160`).

DE CE: la linia 155 ai un `def ogl_numar(...)` indentat cu 4 spații, adică **înăuntrul** lui `test_sterge_cifra` . Indentarea e blocul (B1 din runda 1): tot ce e împins la dreapta sub `def test_sterge_cifra():` aparține corpului lui. Deci `ogl = 0` de la linia 156 e corpul funcției din funcție — care nu e apelată niciodată. Iar `while` de la 158 e din nou la 4 spații, deci e corpul lui `test_sterge_cifra` : se execută după print-uri. Când ajunge la `ogl = ogl * 10 + uc` , Python caută `ogl` printre variabilele locale ale lui `test_sterge_cifra` — numele e local (fiindcă în funcție i

se atribuie o valoare), dar încă n-a primit niciuna. De unde să ia? `ogl = 0` e cu un nivel mai adânc, în altă funcție.

Și de ce merge totuși `palindrom` : `ogl_numar` de la linia 42, cel de la nivelul fișierului, e o funcție diferită și intactă. Copia dinăuntru nu suprascrie nimic — doar strică funcția în care a nimerit.

Cum arată acum, cu indentarea din fișier — ea e tot bug-ul:

```
def test_sterge_cifra()->None:
    print("n=")
    n=int(input())
    print("c=")
    c=int(input())
    print(sterge_cifra(n,c))
    print(sterge_cifra_py(n,c))

    def ogl_numar(n:int)->int:   # <- 4 spații = înăuntrul funcției
     ogl=0                       # <- corpul ei; nu se execută nicicând

    while(n!=0):                # <- 4 spații = corpul lui test_...
        uc=n%10
        ogl=ogl*10+uc           # <- crapă: ogl n-are valoare AICI
        n=n//10

    return ogl
```

FIX: șterge liniile 155-163. `ogl_numar` îl ai deja scris corect la linia 42.

B2 🔴 `numar_cifre(0)` întoarce 0, iar stilul B întoarce 1 — `cifre.py:25–30` · NEATINS din runda 1 (era B5)

CUM E ACUM: `numar_cifre(0)` → **0**, `nr_cifrelor_py(0)` → **1**. Fișa, problema 2, spune explicit: „Pentru n = 0 răspunsul e 1".

DE CE: `while n != 0` verifică condiția **înainte** de primul pas. Pentru `n = 0` condiția e falsă imediat, corpul nu se execută niciodată și `nr` rămâne cu valoarea de start, 0. În C++ reparai asta cu `do { ... } while (n != 0);`, care execută întâi și verifică după. Python **n-are** `do...while` — deci cazul se tratează pe față, înainte de buclă.

Partea importantă: aici stilul B ți-a spus singur că stilul A e greșit. `len(str(0))` e 1, pentru că textul `"0"` are un caracter. Când cele două stiluri dau răspunsuri diferite pe aceeași intrare, una din ele e

sigur greșită — de asta fișa cere ambele versiuni și le afișează una lângă alta, cu `print(numar_cifre(n), numar_cifre_py(n))`. Un singur `print` cu `n = 0` în `main.py` și bug-ul ieșea la prima rulare, fără mine.

FIX: `if n == 0: return 1` înaintea buclei.

🟡 Importante

M1 🟡 **Numele funcțiilor nu sunt cele cerute de fișă** — `cifre.py:32, 42, 51` `nr_cifrelor_py` → `numar_cifre_py`, `ogl_numar` → `oglindit`, `palindrom` → `este_palindrom`. Lipsește de tot `oglindit_py` (problema 3 cere și stilul B pentru oglindit). Numele din fișă nu sunt un capriciu: `main.py` din fișă, soluția mea și review-ul următor le folosesc — dacă diferă, nu se mai potrivește nimic.

M2 🟡 **Problema 4 e pe jumătate, și tipul întors e greșit** — `cifre.py:58–61` `cifra_maxima` (stilul A, numai cu `%` și `//`) nu există. `cifra_maxima_py` e antetată `-> tuple[int, int]`, dar `return [int(m), s.count(m)]` întoarce o **listă**. `m, k = ...` merge și cu listă, dar antetul tău minte — paranteze rotunde: `return (int(m), s.count(m))`.

M3 🟡 **`este_prim_py` doar apelează `este_prim`** — `cifre.py:177–178` Stilul B nu e scris, e redirecționat. Scrie-l cu uneltele din teorie (`all(...)`, `range`) sau măcar cu `for ... in range(2, ...)`.

M4 🟡 **`main.py` importă 7 funcții și apelează una** — `main.py:2–21` Fișa cere `def main() -> None:` și apoi `if __name__ == "__main__": main()`. La tine codul stă direct în `if`, indentat cu 9 spații, și testează doar problema 7 — restul funcțiilor `test_*` sunt importate degeaba. De-aia B1 și B2 n-au fost văzute: codul care le-ar fi scos la iveală nu rulează.

M5 🟡 **`__pycache__/` e tot comis** — 2 fișiere `.pyc` în `git ls-files` · **NEATINS din runda 1 (M1)** `.gitignore` cu `__pycache__/` și `.venv/`, apoi `git rm -r --cached __pycache__`.

M6 🟡 **VS Code rulează tot pe Python-ul de sistem** — `.vscode/settings.json:2` · **NEATINS din runda 1 (M2)** `"ms-python.python:system"`. Ctrl+Shift+P → *Python: Select Interpreter* → `.venv`.

🟢 Cleanups

C1 🟢 **Paranteze inutile** — `cifre.py:4, 27, 44, 66, 71, 83, 122, 170` `while(n!=0):` → `while n != 0:`; `if(palindrom(n)):` → `if este_palindrom(n):`.

C2 🟢 **Spații și indentare** — `cifre.py:3, 12, 33, 156` Spațiu în plus după `s=0`; linia 12 indentată cu 3 spații, 156 cu 5; `str (n)` → `str(n)`. 4 spații, mereu.

C3 🟢 **Comentariile de lucru** — `cifre.py:9, 31`, `main.py:15` `#### 1234 -> "1234" = "4" => 4` sunt notițele tale din timpul lecției. Scoase sau rescrise ca o linie care spune ce face funcția.

Before / After

Pentru B1 vezi blocul de cod de mai sus: în tabel indentarea nu se vede, și tocmai indentarea e problema.

	Acum	Corect

B2	`nr=0`
	`while(n!=0):`
	`nr+=1`
		`if n == 0:`
		`return 1`
		`nr = 0`
		`while n != 0:`
		`nr += 1`

M2	`return [int(m),s.count(m)]`	`return (int(m), s.count(m))`

M4	`if __name__=="__main__":`
	`n=int(input())`
		`def main() -> None:`
		`n = int(input())`
		
		`if __name__ == "__main__":`
		`main()`

Q&A

1. De ce `palindrom(121)` funcționează, deși `ogl_numar` apare de două ori în fișier? Care dintre cele două se apelează, și de unde știi?
2. Fișa cere ca stilul A și stilul B să dea mereu aceeași valoare. Scrie în `main()` o buclă `for n in range(0, 200)` care compară `numar_cifre(n)` cu `numar_cifre_py(n)` și afișează doar valorile unde diferă. Câte găsești înainte de fix? Câte după?
3. `cifre_pare(1357)` întoarce `-1`. Ce valoare are `p` în momentul acela și de ce exact `p == 1` e condiția corectă de „n-am păstrat nicio cifră”?

Runda 1 — 2026-10-27 — sha 6158545

- ✅ **B1** corpul lui `while` neindentat (`IndentationError`) — închis
- ✅ **B2** `nr++` nu există în Python — închis, scrie `nr += 1`
- ✅ **B3** `/` dă `float`, `numar_cifre(1234)` = 327 — închis, folosește `//`
- ✅ **B4** `numar_cifra` vs `numar_cifre` (`NameError`) — închis

- 🔴 **B5** `numar_cifre(0)` întoarce 0 — **NEATINS, vezi B2 din runda 2**
- 🔴 **M1** `__pycache__/` comis — **NEATINS, vezi M5 din runda 2**
- 🔴 **M2** interpretorul VS Code = Python de sistem, nu `.venv` — **NEATINS, vezi M6 din runda 2**
- 🟢 C1 spații în jurul operatorilor · 🟢 C2 paranteze la `while` — parțial (vezi C1, C2 runda 2)